Lesson 1: C++ fundamentals

The course uses C++17 as its maximum language standard. Teaching code uses explicit types, equals initialization for scalar values, parentheses for ordinary object construction, and initializer lists after equals for arrays and aggregate data.

These are continuous lecture notes following the original repository's demonstration sequence. Each experiment includes conceptual reasoning, a complete single-file program, a state trace, failure analysis, and a self-check with an answer. A separate bank contains 120 exercises; function and pointer practice follows the original Lesson 2 placement.

Use CPP-Classroom with C++17 and paste one complete .cpp program. The platform executes code but does not perform this course's automatic grading. Prefill stdin or send lines interactively; send EOF for programs that read until end-of-input.

Original repository: unnc-aim/aim-2526-rookie-courses

Contents

- 01. From Python to a C++ program

- 02. Namespaces and first output

- 03. Input, variables, and output formatting

- 04. Evaluating operators step by step

- 05. Conditions and for loops

- 06. Arrays, std::array, and range iteration

- 07. Searching, replacing, inserting, and erasing strings

- 08. Lesson 1 integration and a preview of structures

01 From Python to a C++ program

Original source: README.md; Lesson 01/my_first_cplus.cpp: headers and main

1.1 Source code, compilation, and one complete run

Last year’s course starts from your Python knowledge: variables, conditions, loops, and functions are already familiar. C++ changes their notation and execution model. Braces delimit blocks, most statements end with a semicolon, and the compiler checks types. The goal is to read and modify the team’s C++ projects, rather than translate Python line by line.

CPP-Classroom compiles the complete source file for the selected exercise. The compiler checks syntax and types before producing an executable, which the platform then runs. Running again does not resume the previous execution or retain its local variables. Each execution starts again in main.

The course uses C++17. Each complete example includes the required standard headers and one main, so you can paste the whole program into the platform. Do not combine two programs that both define main, or paste a g++ shell command into the C++ editor. Local compiler commands are for instructor verification; students use the platform’s Run control.

Complete classroom experiment

`L1_C01_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

Execution trace

After compilation, main executes the output statement, writes one line, and returns 0. The return value is the process exit status, so it does not appear in standard output.

Common error and its cause

Read the first diagnostic and its line number after compilation fails. One missing semicolon can produce errors on several later lines.

Self-check

If you remove the semicolon after the output statement, does the program run before reporting the error?

Answer: No. Compilation fails, so that source version is not executed.

1.2 Headers, main, and statement blocks

The original example includes iomanip, iostream, cmath, array, and string. Each header provides declarations: streams, formatting manipulators, mathematical functions, or library types. An include does not run another program; it makes the required interfaces available to the compiler.

A conventional console program enters through main. Its int return type specifies an integer exit status, parentheses contain its parameter list, and braces enclose its body. Additional braces can introduce local scopes inside the function. Ordinary local variables cease to exist when their scope ends.

The original #ifdef _WIN32 and chcp 65001 adapt certain Windows terminals. CPP-Classroom runs programs on a separate Linux node, so classroom versions do not call system or wait for an external shell. The notes retain Chinese explanations, while basic machine-checked samples use deterministic ASCII output.

Complete classroom experiment

`L1_C01_D02.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int value = 3;
    {
        int doubled = value * 2;
        std::cout << doubled << '\n';
    }
    std::cout << value << '\n';
    return 0;
}
```

Execution trace

The outer value remains alive, but doubled is valid only in the inner block. The output is 6 followed by 3. Referring to doubled on the last line causes a compilation error.

Common error and its cause

Do not rely on a header being included indirectly on one toolchain. Complete examples explicitly include the interfaces they use.

Self-check

Why can a source file not define two main functions?

Answer: It would define the program entry point twice.

Chapter exercises

ID	Exercise
S08	Identify failure stages

02 Namespaces and first output

Original source: Lesson 01/stupid_namespace.cpp; new_new

2.1 std, using, and name lookup

The original stupid_namespace.cpp demonstrates name conflicts. Standard library names such as cout, cin, and string belong to the std namespace. std::cout explicitly selects cout from std; the double colon is the scope resolution operator. The intuition resembles a Python module prefix, but the language mechanisms differ.

using namespace std permits unqualified lookup of standard names. This shortens expressions but increases the scope for collisions. The original mygo namespace defines its own cout and cin and even a struct named std. A bare cout can become ambiguous, and so can std itself. Understand the lookup problem instead of treating either style as an absolute rule.

The original resolves the conflict with ::std::cout, starting explicitly from global scope. Our complete examples generally use std:: while retaining the original collision experiment. In a larger project, avoid global using directives in public headers because they also affect including files.

Complete classroom experiment

`L1_C02_D01.cpp` · CPP-Classroom source

```
#include <iostream>

namespace aim {
int cout = 310;
}

int main() {
    std::cout << aim::cout << '\n';
    ::std::cout << "standard output\n";
    return 0;
}
```

Execution trace

aim::cout is our integer 310, whereas std::cout is an output stream. Their names merely share a spelling. The first statement prints the integer through the stream; the second prints text.

Common error and its cause

A namespace is not an object. The :: selects a name; the following << inserts content into the stream.

Self-check

If mygo contains a type named std, why can ::std::cout still identify the standard library?

Answer: The leading :: starts lookup in global scope and avoids the imported type with the same name.

2.2 cout, newlines, and flushing

The original new_new prints a Hello, C++! message. This introduces the pattern used for later output: std::cout on the left and a value inserted into the stream on the right. Chained << operations combine text, integers, and other printable values in order.

The escape sequence backslash-n adds a newline. std::endl adds a newline and flushes the output buffer, so they are not identical. Prefer a newline for ordinary batch output. Flushing matters when an interactive prompt must be delivered before waiting for input. endl is not a required line-ending keyword.

Single quotes denote a character literal, such as a newline character; double quotes denote a string literal. Spaces are part of the result. Compare field order, separators, and line endings instead of checking only whether the numbers look right.

Complete classroom experiment

`L1_C02_D02.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int id = 7;
    std::cout << "Hello, C++!\n";
    std::cout << "id=" << id << '\n';
    return 0;
}
```

Execution trace

The two output lines are Hello, C++! and id=7. The string id= does not interpolate the variable; another insertion outputs id.

Common error and its cause

std::cout << "id" prints the two letters and does not evaluate the variable id.

Self-check

If the newline after the id output is removed and DONE is then printed, how many lines appear?

Answer: Hello remains on the first line; id=7 and DONE share the second line unless another separator is inserted.

Chapter exercises

ID	Exercise
S01	Hello and exit status
S02	Output with multiple fields
S07	Resolve namespace collisions

03 Input, variables, and output formatting

Original source: input_int; input_variable

3.1 Integer input and extraction order

The original input_int declares x and y before reading cin >> x >> y. C++ needs their types before extraction. int x = 0 initializes x to zero; it does not guarantee successful input. Initialization and validation solve different problems.

Formatted extraction skips leading whitespace. The numbers 8 and 4 can be separated by spaces, tabs, or line breaks. The text 8,4 is different: a comma is not a whitespace separator, so the second extraction fails.

The platform closes stdin after sending prefilled input, which suits a known number of values. Interactive input sends lines while the program runs. A read-until-end loop needs EOF when input is complete. EOF is a stream condition, not the three letters E, O, and F.

Complete classroom experiment

`L1_C03_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int x = 0, y = 0;
    if (!(std::cin >> x >> y)) {
        std::cout << "INVALID\n";
        return 1;
    }
    std::cout << x + y << ' ' << x - y << '\n';
    return 0;
}
```

Standard input:

```
8 4
```

Execution trace

With input 8 4, the stream assigns x first and y second, then prints 12 4. With 8 robot, the second extraction fails. The program reports invalid input rather than treating the initialized zero in y as a valid reading.

Common error and its cause

Newlines in the source editor organize code. Newlines in stdin are data for the current execution.

Self-check

Does entering 8 and 4 on separate lines produce the same result as separating them with a space?

Answer: Yes. Both inputs supply the same two valid integers.

3.2 double, char, string, and formatting state

The original input_variable reads a floating-point value a, a character b, and a string c. double represents approximate real values, char represents a character, and std::string manages a sequence of characters. Declarations determine types, not variable names. A character A and a string A also have different syntax and types.

fixed and setprecision(2) cause subsequent floating-point output to use two decimal places. They change formatting, not the stored value. The formatting state persists until changed, so a later display difference may come from the stream state rather than the calculation.

Reading a string with >> stops at whitespace. Use getline for a name such as AIM Sentry. A newline may remain after extracting a number, causing the next getline to read an empty line. getline(cin >> ws, name) works when leading whitespace is not meaningful; otherwise handle the separator more selectively.

Complete classroom experiment

`L1_C03_D02.cpp` · CPP-Classroom source

```
#include <iomanip>
#include <iostream>
#include <string>

int main() {
    double a = 0.0;
    char b = '\0';
    std::string c;
    std::cin >> a >> b >> c;
    std::cout << std::fixed << std::setprecision(2) << a << ' ' << b << ' ' << c
              << '\n';
    return 0;
}
```

Standard input:

```
3.14159 A mutsumi
```

Execution trace

Input 3.14159 A mutsumi produces 3.14 A mutsumi. c receives one whitespace-delimited word. Any additional word remains unread in the stream.

Common error and its cause

Without fixed, setprecision(2) normally controls significant digits rather than two digits after the decimal point.

Self-check

After printing two decimal places, has a itself been reduced to two decimal places?

Answer: No. Formatting changes the display; later calculations still use the stored double value.

Chapter exercises

ID	Exercise
S03	Read two integers
S04	Read a name containing spaces
S05	Read a line after a number
S06	Detect failed input

04 Evaluating operators step by step

Original source: calculate, in its original internal order

4.1 Remainders, integer division, and conversion timing

The original calculate compares 3%2, 3/2, and 3.14/2. The remainder is 1. Dividing two ints performs integer division, so 3/2 is 1. A double operand changes the operation to floating-point division, giving approximately 1.57. Operand types determine the computation before output formatting occurs.

Convert at least one operand before division to obtain the real quotient. static_cast<double>(3/2) computes the integer result 1 first and only then converts it to 1.0. Assigning a result to double cannot restore information lost in an earlier integer operation.

Integer division truncates toward zero, so -3/2 is -1. This differs from Python's floor division. Validate a divisor before dividing by it. Signed integer arithmetic does not grow automatically either: a wide operand such as 1LL can make an intermediate multiplication use long long.

Complete classroom experiment

`L1_C04_D01.cpp` · CPP-Classroom source

```
#include <iomanip>
#include <iostream>

int main() {
    std::cout << 3 % 2 << ' ' << 3 / 2 << '\n';
    std::cout << std::fixed << std::setprecision(2) << static_cast<double>(3) / 2
              << ' ' << static_cast<double>(3 / 2) << '\n';
    return 0;
}
```

Execution trace

The first line is 1 1. On the second line, conversion before division produces 1.50, whereas conversion after integer division produces 1.00.

Step	State / operation	Result
`3 / 2`	`int / int`	`1`
`static_cast<double>(3) / 2`	`double / int`	`1.5`
`static_cast<double>(3 / 2)`	`int division, then conversion`	`1.0`

Common error and its cause

Changing only the destination variable to double does not repair integer division or overflow that already occurred.

Self-check

What is result after double result = 5 / 2?

Answer: It is 2.0 because the right-hand expression uses integer division. Use 5/2.0 or convert an operand to obtain 2.5.

4.2 Assignment, comparison, and bool output

The original sets x=10 and y=1, compares them, then assigns y=x. A single equals sign assigns the right-hand value to the object on the left; a double equals sign compares equality. Assignment changes state, while comparison produces a bool. Both can appear in expressions, which makes accidental assignment easy to miss.

A bool has the values true and false. Streams normally display them as 1 and 0; boolalpha displays the words instead. Comparisons such as x<=y or x!=y produce bool values. Naming intermediate conditions makes a larger decision easier to inspect.

Do not chain mathematical inequalities directly. In C++, 0<=x<=100 first computes a bool and then compares that bool with 100. Write 0<=x && x<=100 instead. As in the original demonstration, evaluate each output using the values that exist at that point in execution.

Complete classroom experiment

`L1_C04_D02.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int x = 10, y = 1;
    std::cout << std::boolalpha << (x == y) << '\n';
    y = x;
    std::cout << (x == y) << ' ' << (y != x) << '\n';
    return 0;
}
```

Execution trace

The first comparison is false. Assignment changes y from 1 to 10, so the next line is true false. The assignment does not change x.

Step	State / operation	Result
initial	x=10	y=1

Step	State / operation	Result
y=x	x=10	y=10
comparison	x==y: true	y!=x: false

Common error and its cause

if (x=0) assigns zero to x and then tests the assigned value. Compiler warnings help, but understanding the expression remains necessary.

Self-check

After y=x, will changing x to 3 automatically change y to 3?

Answer: No. Ordinary integer assignment copies the current value and does not create a continuing binding. References are covered in Lesson 2.

4.3 Logical AND, OR, NOT, and short-circuiting

The original combines comparisons with &&, ‖, and !. AND requires both conditions, OR requires at least one, and NOT negates a condition. Read the individual comparisons before combining them. Keep prose and code consistent: the original contains an explanatory line whose inequality does not match its description.

Both && and ‖ short-circuit from left to right. If the left operand determines the result, the right operand is not evaluated. This can guard an operation with a precondition. We first demonstrate it with division, then revisit the same idea for pointer dereferencing in Lesson 2.

Use parentheses to make compound rules explicit, especially when ! interacts with comparisons. For a firing gate, identify the health, heat, and target conditions separately before joining them. This makes boundary review easier than reading an unexplained chain of operators.

Complete classroom experiment

`L1_C04_D03.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int a = 9, b = 0;
    bool valid = b != 0 && a / b > 2;
    std::cout << std::boolalpha << valid << '\n';
    std::cout << !(3 > 1 && 3 < 5) << '\n';
    return 0;
}
```

Execution trace

Because b!=0 is false, a/b is not evaluated, and the first line safely prints false. The condition in parentheses on the second line is true, so its negation is false.

Common error and its cause

A guard placed after the unsafe operation cannot protect it: a/b>2 && b!=0 attempts division first.

Self-check

Does false && expensive() call expensive?

Answer: No. The left operand already determines the result of the AND expression.

4.4 Math functions, increment, and compound assignment

The latter half of calculate demonstrates sqrt, pow, ceil, floor, then increment and compound assignment. sqrt returns an approximate square root; pow computes a power; ceil and floor round toward positive and negative infinity. Their results are floating-point values, despite representing integral mathematical values.

Negative values distinguish the rounding rules: ceil(-2.5) is -2, floor(-2.5) is -3, and conversion to an integer normally truncates to -2. Floor does not simply remove the fractional part. Floating-point measurements may also need a tolerance defined by the problem.

x+=5 adds five and stores the result in x. As standalone statements, ++x and x++ both increment x. Their expression values differ: one provides the updated value and the other the previous value. Keep updates separate while learning rather than reading and modifying one variable repeatedly in a complicated expression.

Complete classroom experiment

`L1_C04_D04.cpp` · CPP-Classroom source

```
#include <cmath>
#include <iostream>

int main() {
    std::cout << std::ceil(-2.5) << ' ' << std::floor(-2.5) << '\n';
    int x = 10;
    x++;
    x += 5;
    x--;
    std::cout << x << '\n';
    return 0;
}
```

Execution trace

The first line is -2 -3. The successive values of x are 10, 11, 16, and 15. Trace state after each statement rather than guessing from the number of operators.

Step	State / operation	Result
`initial`	`x`	`10`
`x++`	`x`	`11`
`x+=5`	`x`	`16`
`x--`	`x`	`15`

Common error and its cause

For a small integer square, x*x is clear and can stay in integer arithmetic. Do not assume pow provides an exact integer result or eliminates range concerns.

Self-check

Are floor(-1.2) and static_cast<int>(-1.2) equal?

Answer: No. The former is -2 and the latter is -1.

Chapter exercises

ID	Exercise
S09	Integer and floating-point division
S10	Split a duration
S11	Average three measurements
S12	Multiply large integers
S13	Convert Celsius to Fahrenheit
S14	Distance from the origin
S15	Compare with an absolute tolerance
S16	Ceiling division for nonnegative values

05 Conditions and for loops

Original source: if_and_for

5.1 if, else if, and mutually exclusive branches

The original if_and_for classifies an input integer as positive, negative, or zero. The condition belongs in parentheses, and braces enclose the selected statements. An else-if condition is considered only after earlier conditions fail; the final else handles the remainder.

Branch order expresses priority. If zero health must produce DEAD, handle that special case before a broader low-health condition. Separate if statements are not mutually exclusive and may trigger multiple actions. Write the ranges or priority rules first, then translate them into branches.

Braces help preserve intended control when a branch later gains another statement. Without them, an added statement may execute unconditionally. Test a positive value, a negative value, and zero: two positive examples do not cover three branches.

Complete classroom experiment

`L1_C05_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int x = 0;
    std::cin >> x;
    if (x > 0) {
        std::cout << "POSITIVE\n";
    } else if (x < 0) {
        std::cout << "NEGATIVE\n";
    } else {
        std::cout << "ZERO\n";
    }
    return 0;
}
```

Standard input:

```
0
```

Execution trace

For input 0, both comparisons are false, so only the final ZERO branch executes.

Common error and its cause

A semicolon immediately after if (x>0) creates an empty controlled statement; a following block is then separate.

Self-check

If else if becomes a separate if, which if owns the final else?

Answer: It belongs to the nearest unmatched if, which can change the intended three-way decision.

5.2 The initialization, condition, and update of for

The original prints five iterations for i=1 through i=5. The three clauses initialize state, test continuation, and update after each iteration. Initialization runs once, followed by repeated condition, body, and update steps. An initially false condition gives zero iterations.

Counting iterations and indexing an array use different endpoints. Five printed iterations can use 1..5, but a five-element array has indexes 0..4, normally expressed as i<5. The half-open range [0,n) contains exactly n indexes.

Initialize an accumulator outside the loop. After each iteration it represents the portion already processed, providing an invariant for reasoning. Inputs n=0, n=1, and an ordinary n expose initialization and endpoint mistakes. The exercise bank extends to while, break, and continue after this original for-loop model is established.

Complete classroom experiment

`L1_C05_D02.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int n = 0;
    std::cin >> n;
    int sum = 0;
    for (int i = 1; i <= n; ++i) {
        sum += i;
        std::cout << i << ':' << sum << '\n';
    }
    return 0;
}
```

Standard input:

```
3
```

Execution trace

Input 3 gives 1:1, 2:3, and 3:6. Input 0 produces no lines because the initial condition fails.

Step	State / operation	Result
`iteration 1`	`i=1`	`sum=1`
`iteration 2`	`i=2`	`sum=3`

Step	State / operation	Result
`iteration 3`	`i=3`	`sum=6`
`next condition`	`i=4`	`4<=3: false`

Common error and its cause

Declaring a new int sum = 0 inside the loop resets the accumulator each iteration and loses previous work.

Self-check

How many accesses does i=0; i<=n; ++i make for an array of length n?

Answer: It makes n+1 accesses, including the invalid index n. Use i<n.

Chapter exercises

ID	Exercise
S17	Three health categories
S18	Closed-interval membership
S19	Firing gate
S20	Protect a division
S21	Dispatch with switch
S22	Median of three values
S23	Gregorian leap years
S24	Priority in safety decisions
S25	Accumulate in a loop
S26	Sentinel-controlled input
S27	Skip and stop
S28	Sum decimal digits
S29	Enumerate factors
S30	Primality test
S31	Print a rectangle
S32	Longest consecutive detection run

06 Arrays, std::array, and range iteration

Original source: score; score_new; shuzu

6.1 Fixed arrays and zero initialization

The original score[5]={94,87,50} creates five integers with three explicit initializers. Aggregate initialization zero-initializes the remainder, producing 94, 87, 50, 0, 0. This does not mean every local array is automatically zeroed: an uninitialized local array of fundamental values has no such guarantee.

Square brackets select an array element using a zero-based index. score[0] is first and score[4] is last; score[5] is outside the array. Built-in indexing does not validate bounds. Undefined behavior may show wrong data, a crash, or apparently normal output; the latter does not establish correctness.

The original comment suggests global arrays as a general answer to large data. Storage constraints depend on implementation, platform, and thread-stack configuration. Prefer std::vector for large or runtime-sized data rather than treating shared global state as a universal storage solution.

Complete classroom experiment

`L1_C06_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int main() {
    int score[5] = {94, 87, 50};
    for (int i = 0; i < 5; ++i) {
        std::cout << i << ':' << score[i] << '\n';
    }
    return 0;
}
```

Execution trace

The third explicit value occupies index 2. Indexes 3 and 4 contain zero. The loop runs five times and never accesses index 5.

Step	State / operation	Result
score[0]	explicit	94
score[1]	explicit	87
score[2]	explicit	50
score[3]	value-initialized	0

Step	State / operation	Result
score[4]	value-initialized	0

Common error and its cause

A length of five does not imply a final index of five.

Self-check

If main contains only int score[5];, can you assert that the final two elements are zero?

Answer: No. That declaration provides no zero-initialization guarantee.

6.2 Copying built-in arrays and assigning std::array

The original shuzu copies new_scores into score element by element because built-in arrays do not support whole-array assignment. Their declared extent does not change through assignment. Update elements individually or use a container with whole-object value semantics.

std::array<int,5> packages fixed-size storage as a standard container. It supports size, at, range iteration, and assignment from another object of the same type. array<int,5> and array<int,4> are distinct types. The original score_new={1,2,3,4,5} uses std::array assignment and does not establish the same operation for a built-in array.

at throws on an out-of-range index, whereas [] does not offer the same check. Establish valid indexes first, then study the interface difference. Correct loop bounds are the normal way to prevent an invalid access.

Complete classroom experiment

L1_C06_D02.cpp · CPP-Classroom source

```
#include <array>
#include <iostream>

int main() {
    std::array<int, 5> scores = {94, 87, 50};
    std::array<int, 5> replacement = {80, 85, 90, 95, 100};
    scores = replacement;
    std::cout << scores.size() << ' ' << scores.at(0) << '\n';
    return 0;
}
```

Execution trace

Assignment copies all five elements from replacement into scores, giving 5 80. They remain independent objects: a later change to replacement does not automatically update scores.

Common error and its cause

Do not generalize std::array assignment to built-in arrays.

Self-check

Can push_back change an `array<int,5>` size to six?

Answer: No. It has no growth-oriented push_back operation. Use vector for a changing size.

6.3 Copies and modification in range-based for

The original for(int j:score_new) copies one element into j on each iteration. This works well for printing small integers. Assigning zero to j does not zero the container element because j is a local copy. Lesson 2 formalizes the reference syntax used to change that behavior.

Use int& j when you intend to modify the original element. const std::string& is a common read-only choice for larger objects because it avoids copying and prevents mutation through the reference. The original Lesson 2 uses Band& with `vector<Band>`; both examples answer the same question about copies versus original objects.

vector can append elements at runtime, making it suitable for an unknown input count. Growth may reallocate storage and invalidate references, pointers, or iterators to existing elements. This extends the original array lesson and is explained before exercises rely on dynamic containers.

Complete classroom experiment

`L1_C06_D03.cpp` · CPP-Classroom source

```
#include <array>
#include <iostream>

int main() {
    std::array<int, 3> a = {1, 2, 3};
    for (int value : a) {
        value = 0;
    }
    for (int &value : a) {
        value *= 2;
    }
    for (int value : a) {
        std::cout << value << '\n';
    }
    return 0;
}
```

Execution trace

The first loop changes only copies, leaving a as 1, 2, 3. The second changes the elements to 2, 4, 6, which the final loop prints.

Common error and its cause

Do not retain a vector element reference across growth and assume it is still valid.

Self-check

What is printed if the & is removed from the second loop?

Answer: 1, 2, 3, because both update loops would modify only copies.

Chapter exercises

ID	Exercise
S33	Initialize std::array
S34	Maximum element
S35	Append to a vector
S36	Validate an index
S37	Copy and modify a vector
S38	Adjacent differences
S39	Stable filtering
S40	Rotate right

07 Searching, replacing, inserting, and erasing strings

Original source: str_modern, retaining the original member-name context

7.1 Strings and safe character access

The original mygo="soyo,anon,taki,rana,tomori" ties the string operations together. std::string manages a sequence whose length can change without manually reserving a C character buffer. The commas are ordinary characters, not automatically a collection of five member objects.

at(3) selects the fourth character, which is o in the initial text. Check empty before using front or back. size counts stored char units; for UTF-8 text, that byte count is not the number of Chinese characters.

The same variable holds different text after each operation. We track its current value at every step, correcting stale output comments in the original. During practice, inspect the result after each replace, insert, or erase.

Complete classroom experiment

`L1_C07_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string mygo = "soyo,anon,taki,rana,tomori";
    std::cout << mygo.at(3) << '\n';
    std::cout << mygo.size() << '\n';
    return 0;
}
```

Execution trace

The initial string has 26 ASCII characters, including commas. at(3) prints o. No automatic member splitting occurs.

Common error and its cause

A character index is not a member index. at(3) does not select the fourth band member.

Self-check

Why can a Chinese string have a size larger than its visible character count?

Answer: A Chinese character usually occupies several UTF-8 bytes, and size counts char units rather than user-perceived characters.

7.2 find and npos

find("anon") returns the starting position of the first match. Four characters in soyo plus one comma place anon at index 5. The original comment claiming position 10 does not match the text; this version derives the position from the actual sequence.

When no match exists, find returns std::string::npos, a special size_type value. It is not a valid index. Preserve the result type with std::string::size_type and compare it with npos before using it in substr, replace, or erase, rather than narrowing to int and assuming -1.

find searches substrings, not comma-delimited member identities. A short query may match within a longer name. Exact field matching requires explicit delimiter rules and parsing.

Complete classroom experiment

`L1_C07_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string mygo = "soyo,anon,taki,rana,tomori";
    std::string::size_type pos = mygo.find("anon");
    if (pos != std::string::npos) {
        std::cout << pos << '\n';
    }
    std::cout << std::boolalpha << (mygo.find("saki") == std::string::npos) << '\n';
    return 0;
}
```

Execution trace

The output is 5 followed by true because saki is absent. Searching does not modify mygo.

Step	State / operation	Result
soyo	indexes	0..3
,	index	4
anon	indexes	5..8
find("anon")	position	5
find("saki")	not found	npos

Common error and its cause

Position zero is a valid successful result. if(pos) is not a success check.

Self-check

If find("anon") returns zero in "anon,soyo", does that mean failure?

Answer: No. Zero means the match starts at the first character; npos indicates failure.

7.3 replace and changes in length

The original replaces anon with saki. In replace(pos,count,text), count is the number of original characters removed, not an ending index. Removing four characters starting at five deletes anon. Replacement text may have a different length, changing the string size.

The original directly passes find("anon") into replace. If anon is absent, npos is not a valid replacement position. The revised example checks the search result first, preserving the original search-then-edit logic while fixing the missing case.

Replacement can shift all later positions. If you cache multiple positions and edit earlier text, later positions may become stale. Re-search the current text after each edit until you have a reason to use a more advanced strategy.

Complete classroom experiment

`L1_C07_D03.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string mygo = "soyo,anon,taki,rana,tomori";
    std::string::size_type pos = mygo.find("anon");
    if (pos != std::string::npos) {
        mygo.replace(pos, 4, "saki");
    }
    std::cout << mygo << '\n';
    return 0;
}
```

Execution trace

The result is soyo,saki,taki,rana,tomori. The total length happens to remain unchanged because both names have four characters.

Common error and its cause

A count of four means four characters; it is not an absolute final index.

Self-check

How far does taki move left if anon is replaced with A?

Answer: Three positions, because four characters are removed and one is inserted.

7.4 Concatenation, insert, erase, and clear

The original next appends with += and inserts text at position zero. += appends at the end; insert specifies an insertion point. Code must also maintain the commas in a comma-separated list: string does not add member separators automatically.

Erasing only the four letters of saki leaves adjacent commas. The original removes five characters to include the following comma. That depends on saki actually having a following separator and cannot be applied blindly to the last field.

clear makes the string empty and size becomes zero. It does not destroy the string object or promise that all allocated storage is returned immediately. Distinguish an empty value from an ended object lifetime; Lesson 2 builds on that distinction.

Complete classroom experiment

`L1_C07_D04.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string text = "soyo,saki,taki";
    text += ",rana";
    text.insert(0, "anon,");
    std::string::size_type pos = text.find("saki,");
    if (pos != std::string::npos) {
        text.erase(pos, 5);
    }
    std::cout << text << '\n';
    text.clear();
    std::cout << text.size() << '\n';
    return 0;
}
```

Execution trace

Appending rana, prepending anon, and removing saki with its following comma yields anon,soyo,taki,rana. After clear, the size output is zero.

Common error and its cause

Do not erase after a failed search. Review separators separately for first, middle, and last fields.

Self-check

Can text += "anon" be used after clear?

Answer: Yes. The object still exists; only its current contents are empty.

Chapter exercises

ID	Exercise
S41	String length and endpoints
S42	Find a substring
S43	Replace the first match safely
S44	Split comma-delimited fields
S45	Uppercase ASCII letters
S46	Count whitespace-delimited words
S47	Parse one integer strictly
S48	Palindrome text

08 Lesson 1 integration and a preview of structures

Original source: The struct preview in the original README; functions remain in Lesson 2

8.1 From parallel variables to one record

The original README lists structures in Lesson 1 without a separate developed example. This minimal bridge groups a robot's identifier, health, and name in one struct instead of maintaining parallel arrays. It also provides the transition to class used in Lesson 2.

The dot operator selects a member of one object. Frame a and Frame b are distinct objects, so changing a.hp does not change b.hp. Band instances revisit this idea in Lesson 2. For now, fields are public; the full access-control lesson follows later.

Lesson 1 closes by combining input, calculation, decisions, iteration, and output. The wider exercise bank includes parsing and simulation, but advanced algorithms and object-management topics must not reorder the original core. Lesson 2 starts with test_add, then references, pointers, swapping, and classes.

Complete classroom experiment

`L1_C08_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

struct Frame {
    int id = 0;
    int hp = 0;
    std::string name;
};

int main() {
    Frame a = {7, 80, "Sentry"};
    Frame b = a;
    a.hp = 30;
    std::cout << a.id << ':' << a.hp << '\n';
    std::cout << b.id << ':' << b.hp << '\n';
    return 0;
}
```

Execution trace

b copies the record's current values. Changing a later gives outputs 7:30 and 7:80.

Common error and its cause

A structure definition ends with a semicolon even though its body uses braces.

Self-check

For five robots, should you use five groups of separate variables or `vector<Frame>`?

Answer: Use a container of Frame records and iterate over objects. A fixed-size collection can also use `array<Frame,5>`.

Chapter exercises

ID	Exercise
S65	A telemetry structure
S66	Select the nearest target
S67	Scoped direction enumeration
S68	Row sums in a fixed matrix
S69	Update a 2D position
S70	Fixed-window means
S71	Deduplicate timestamps
S72	Sorting with tie-breaking
S73	Statistics on valid readings
S74	Run-length encoding
S75	A restricted command interpreter
S76	Heat simulation
S77	Grid collisions
S78	Dropped-frame alarms
S79	Debug a loop endpoint
S80	An integrated unit report